

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

Алгоритми та методи обчислень

Лабораторна робота №2

«Обчислювальна складність алгоритмів сортування»

Виконав: студент
групи ІО-42
Куліков М. М.
Номер у списку групи: 9
Перевірив:
Порєв В. М.

Тема: «Обчислювальна складність алгоритмів сортування».

Мета: Закріплення навичок практичної оцінки алгоритмічної складності логічних алгоритмів на прикладі алгоритмів сортування.

Загальне завдання:

- Написати програму сортування алгоритмом сортування у відповідності до варіанту.
- У програмі передбачити вимірювання часу сортування з максимально можливою точністю.
- Створити 10 тестових масивів даних різного розміру, які підлягають сортуванню.
- Побудувати графік залежності часу виконання алгоритму від розміру вхідного масиву даних.
- Побудувати графік теоретично відомої обчислювальної складності.
- Порівняти одержані графіки часової складності та обчислювальної складності вашого алгоритму сортування.

Вимоги до програмного забезпечення:

- Введення даних з клавіатури і з зовнішнього файлу;
- Перевірка коректності введених даних;
- Інтерфейс у стилі GUI;
- Вимірювання виконувати з кроком 1000 елементів масиву;
- Порівняти графіки залежності у одиницях часу та у кількості виконаних операцій

Індивідуальне завдання:

Завдання виконане згідно 9 варіанту.

Таблиця 1 – Завдання згідно варіанту

Варіант	Назва алгоритму	Алгоритмічна складність
9	Сортування шейкером	$O(n^2)$

Блок-схеми алгоритмів:

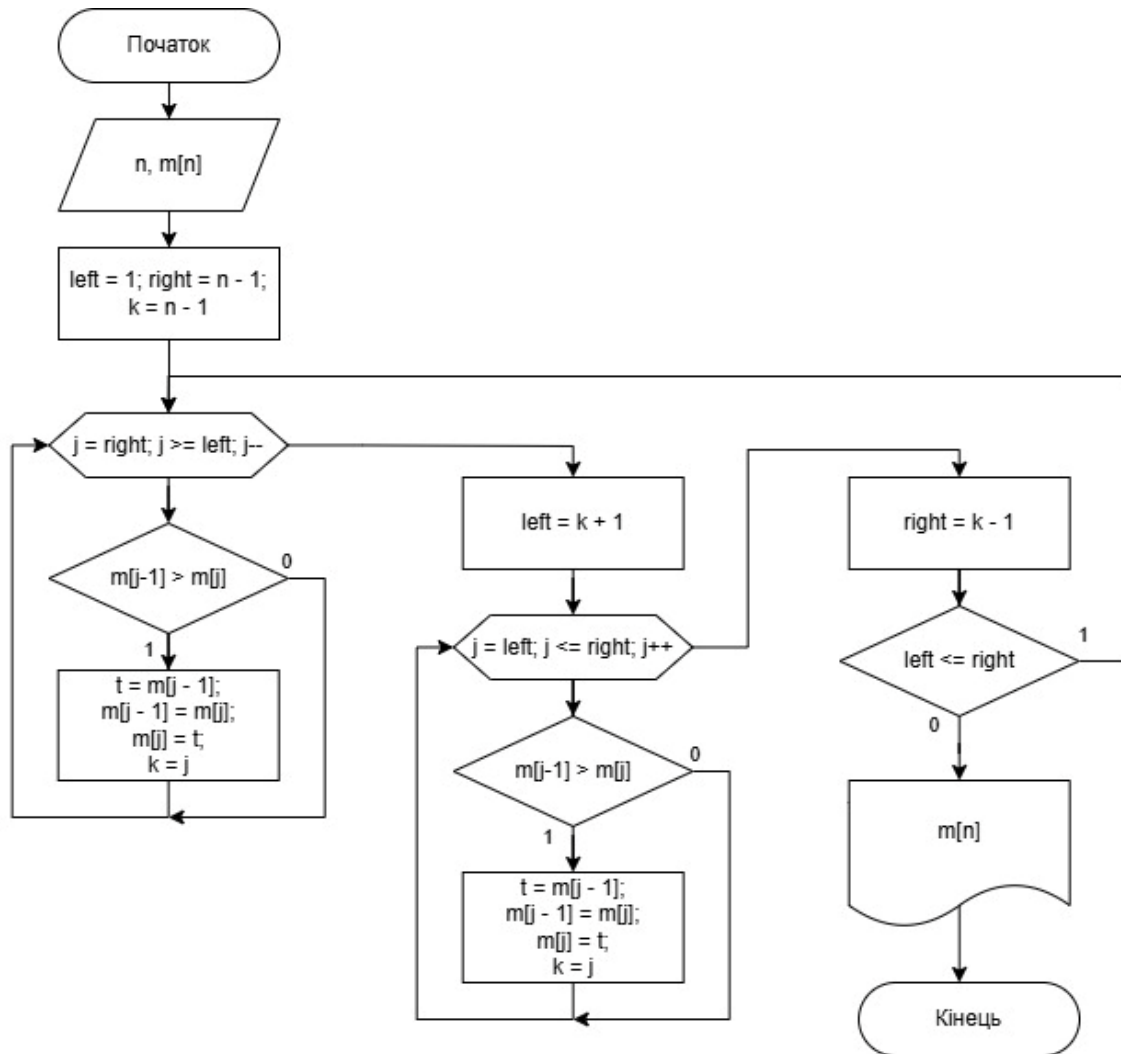


Рисунок 1 – Блок схеми алгоритму сортування шейкером

Код програми:

Оскільки програма написана на WPF, і код є громіздким, то вміст усіх файлів представлено в додатку А. У цьому розділі наведений код для реалізації сортування шейкером.

Вміст файлу ShakerSorter.cs:

```
public class ShakerSorter : ISorter {
    public SortResult Sort(int[] input) {
        if (input == null || input.Length == 0) throw new ArgumentException("Помилка: Масив порожній.");
        int[] m = (int[])input.Clone();
        long ops = 0;
        int n = m.Length;
        int left = 1, right = n - 1, j, t, k = n - 1;
        Stopwatch sw = Stopwatch.StartNew();
        do {
            for (j = right; j >= left; j--) {
                ops++;
                if (m[j - 1] > m[j]) { t = m[j - 1]; m[j - 1] = m[j]; m[j] = t; k = j; }
            }
            left = k + 1;
            for (j = left; j <= right; j++) {
                ops++;
                if (m[j - 1] > m[j]) { t = m[j - 1]; m[j - 1] = m[j]; m[j] = t; k = j; }
            }
            right = k - 1;
        } while (left <= right);
        return new SortResult(m, ops, sw.Elapsed);
    }
}
```

```

for (j = left; j <= right; j++) {
    ops++;
    if (m[j - 1] > m[j]) { t = m[j - 1]; m[j - 1] = m[j]; m[j] = t; k = j; }
}
right = k - 1;
}
while (left <= right);
sw.Stop();
return new SortResult { TimeMs = sw.Elapsed.TotalMilliseconds, Operations = ops, Data = m };
}

```

Результати виконання програми:

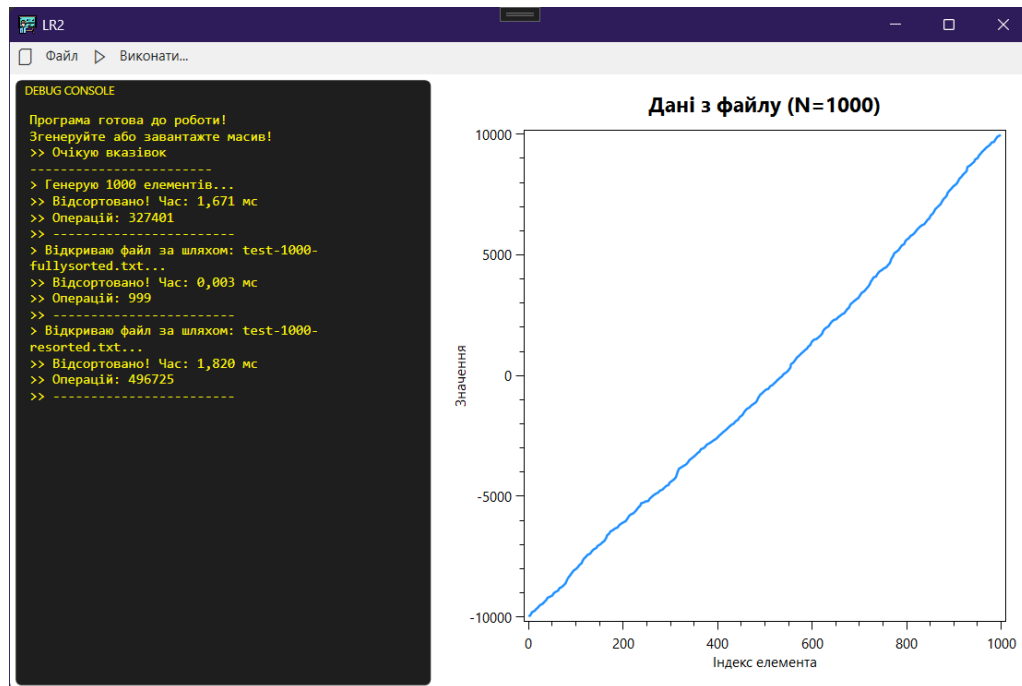


Рисунок 2 – Результат запуску сортування одного масиву

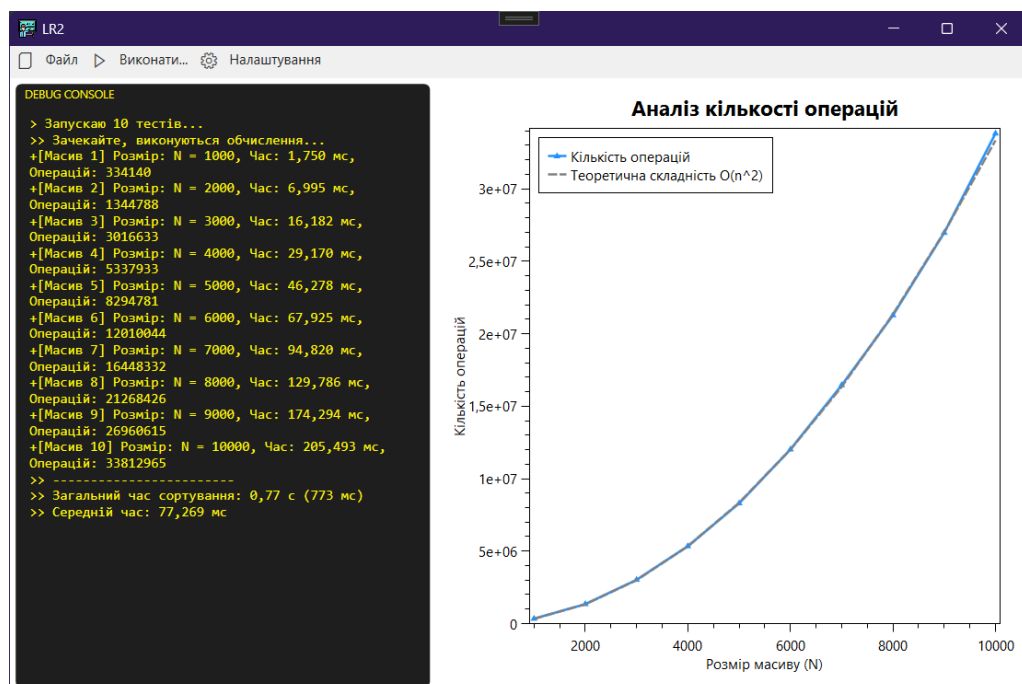


Рисунок 3 – Результат запуску аналізу алгоритму сортування за кількістю операцій

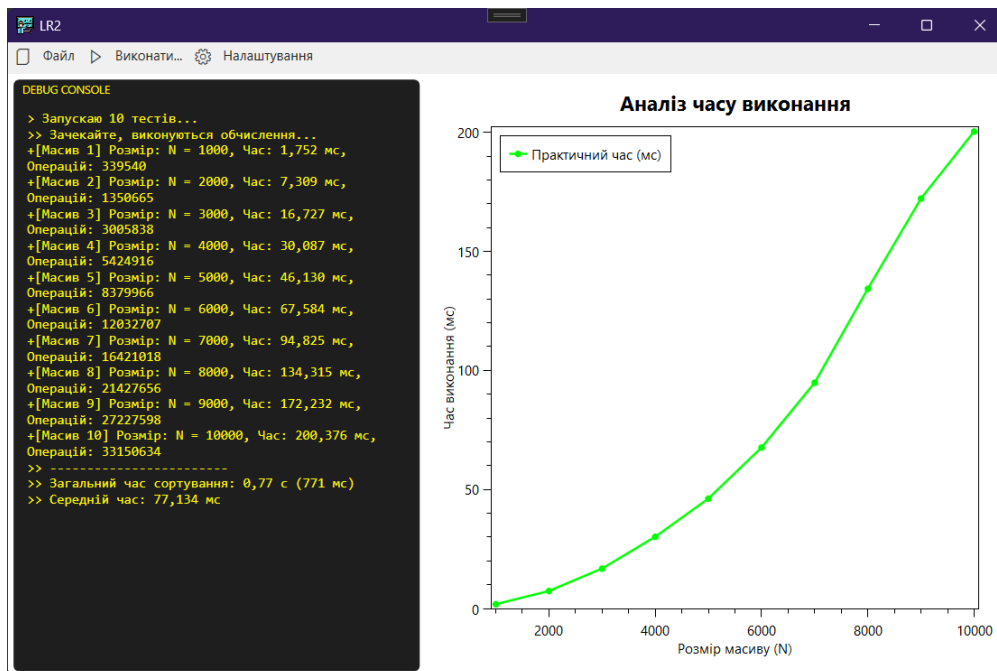


Рисунок 4 – Результат запуску аналізу часу виконання алгоритму сортування

Аналіз результатів:

За результати запусків аналізу, бачимо, що чим більший розмір масиву, тим більше операцій треба виконати для сортування, і, відповідно, час виконання алгоритму значно зростає. Це й узгоджується з теоретичною алгоритмічною складністю $O(n^2)$. Наприклад, для випадково згенерованого масиву з 10 000 елементів програма виконала понад 33 млн операцій, що яскраво підтверджує квадратичну залежність. Час сортування зменшується, якщо подається майже відсортований масив, а коли подається випадковий, то часу на сортування треба значно більше. Також варто враховувати, що час сортування деколи може змінюватись через фонове навантаження.

Висновки:

Отже, під час виконання лабораторної роботи було створено програму, яка реалізовує метод сортування шейкером та досліджено його обчислювальну складність на масивах різних розмірів і з різним початковим станом. Аналіз результатів підтверджує теоретичну часову складність. При введенні випадкових або реверсивних масивів (другий - найгірший сценарій, за якого сортування відбувається найдовше), алгоритм демонструє складність $O(n^2)$. При більших розмірах масивів виконується більше операцій, що веде за собою збільшення часу виконання сортування. Коли ж масиви частково або повністю відсортовані, алгоритм працює значно швидше (і демонструє складність близьку до $O(n)$).

Додаток А

Вміст файлу MainWindow.xaml:

```
<Window x:Class="LR2.MainWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
    xmlns:local="clr-namespace:LR2" xmlns:oxy="clr-namespace:OxyPlot.Wpf;assembly=OxyPlot.Wpf"
    mc:Ignorable="d" Title="LR2" Height="600" Width="900" WindowStartupLocation="CenterScreen" Icon="/icons8-
        mathematics-lineal-color-96.png">

    <DockPanel>
        <Menu DockPanel.Dock="Top" Padding="2">
            <MenuItem Header="Файл">
                <MenuItem.Icon>
                    <TextBlock FontFamily="Segoe MDL2 Assets" Text="&#xE8A5;" FontSize="14" Foreground="#333333"/>
                </MenuItem.Icon>
                <MenuItem Header="Відкрити" Click="LoadFile">
                    <MenuItem.Icon>
                        <TextBlock FontFamily="Segoe MDL2 Assets" Text="&#xE8E5;" FontSize="14" Foreground="#0078D7"/>
                    </MenuItem.Icon>
                </MenuItem>
                <MenuItem Header="Зберегти відсортоване" Click="SaveToFile">
                    <MenuItem.Icon>
                        <TextBlock FontFamily="Segoe MDL2 Assets" Text="&#xE792;" FontSize="14" Foreground="#0078D7"/>
                    </MenuItem.Icon>
                </MenuItem>
                <MenuItem Header="Вихід" Click="ExitApp">
                    <MenuItem.Icon>
                        <TextBlock FontFamily="Segoe MDL2 Assets" Text="&#xE711;" FontSize="14" Foreground="#E81123"/>
                    </MenuItem.Icon>
                </MenuItem>
            </MenuItem>
            <MenuItem Header="Виконати...">
                <MenuItem.Icon>
                    <TextBlock FontFamily="Segoe MDL2 Assets" Text="&#xE768;" FontSize="14" Foreground="#333333"/>
                </MenuItem.Icon>
                <MenuItem Header="створення одного масиву..." Click="OpenInputWindow">
                    <MenuItem.Icon>
                        <TextBlock FontFamily="Segoe MDL2 Assets" Text="&#xE710;" FontSize="14" Foreground="#107C10"/>
                    </MenuItem.Icon>
                </MenuItem>
                <MenuItem Header="повний аналіз..." Click="RunAnalysis">
                    <MenuItem.Icon>
                        <TextBlock FontFamily="Segoe MDL2 Assets" Text="&#xE9D9;" FontSize="14" Foreground="#D83B01"/>
                    </MenuItem.Icon>
                </MenuItem>
            </MenuItem>
            <MenuItem Header="Налаштування">
```

```

        <MenuItem.Icon>
            <TextBlock FontFamily="Segoe MDL2 Assets" Text="&#xE713;" FontSize="14" Foreground="#333333"/>
        </MenuItem.Icon>
        <MenuItem x:Name="mnuShowOperations" Header="Відображати графік операцій" IsCheckable="True"
            IsChecked="True">
            <MenuItem.Icon>
                <TextBlock FontFamily="Segoe MDL2 Assets" Text="&#xE9D9;" FontSize="14" Foreground="#0078D7"/>
            </MenuItem.Icon>
        </MenuItem>
    </MenuItem>
</Menu>

<Grid Margin="5">
    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="360"/>
        <ColumnDefinition Width="*" />
    </Grid.ColumnDefinitions>

    <Border Grid.Column="0" Background="#1E1E1E" CornerRadius="5">
        <DockPanel>
            <TextBlock DockPanel.Dock="Top" Text="  DEBUG  CONSOLE" Foreground="#FFFFFF906" FontSize="10"
                Margin="5,2"/>
            <TextBox x:Name="txtLog" Background="Transparent" Foreground="#FFFFFF906"
                BorderThickness="0" VerticalScrollBarVisibility="Auto" IsReadOnly="True"
                FontFamily="Consolas" FontSize="12" Padding="10" TextWrapping="Wrap"/>
        </DockPanel>
    </Border>

    <oxy:PlotView x:Name="plotView" Grid.Column="1" Background="White" Margin="5,0,0,0"/>
</Grid>
</DockPanel>
</Window>

```

Вміст файлу MainWindow.cs:

```

using LR2.SortingLogic;
using OxyPlot;
using OxyPlot.Axes;
using OxyPlot.Series;
using System.Windows;
using System.Windows.Input;

namespace LR2
{
    public partial class MainWindow : Window
    {
        private readonly SortingManager _appManager;
        public MainWindow()
        {
            InitializeComponent();
            _appManager = new SortingManager(new ShakerSorter());
        }
    }
}

```

```

txtLog.AppendText($"Програма готова до роботи!\n");
txtLog.AppendText($"Згенеруйте або завантажте масив!\n");
txtLog.AppendText($">> Очікую вказівок \n");
txtLog.AppendText($"-----\n");
}

private void SaveToFile(object sender, RoutedEventArgs e)
{
    var dialog = new Microsoft.Win32.SaveFileDialog { Filter = "Text Files (*.txt)|*.txt" };
    if (dialog.ShowDialog() == true) {
        try {
            _appManager.SaveCurrent(dialog.FileName);
            MessageBox.Show("Збережено успішно!");
        }
        catch (Exception ex) { MessageBox.Show(ex.Message); }
    }
}

private void LoadFile(object sender, RoutedEventArgs e)
{
    var dialog = new Microsoft.Win32.OpenFileDialog { Filter = "Text Files|*.txt" };
    if (dialog.ShowDialog() == true) {
        txtLog.AppendText($"> Відкриваю файл за шляхом: {System.IO.Path.GetFileName(dialog.FileName)}...\n");
        Mouse.OverrideCursor = Cursors.Wait;
        Application.Current.Dispatcher.Invoke(() => { }, System.Windows.Threading.DispatcherPriority.Render);

        try
        {
            SortResult res = _appManager.LoadAndSort(dialog.FileName);
            ProcessSingleArray(res, $"Дані з файлу (N={res.Data!.Length})");
        }
        catch (Exception ex)
        {
            txtLog.AppendText($"[!] ПОМИЛКА: {ex.Message}\n");
            txtLog.ScrollToEnd();
        }
        finally { Mouse.OverrideCursor = null; }
    }
}

private void ProcessSingleArray(SortResult result, string title)
{
    txtLog.AppendText($">> Відсортовано! Час: {result.TimeMs:F3} мс\n");
    txtLog.AppendText($">> Операцій: {result.Operations}\n");
    txtLog.AppendText($">> -----\n");

    var model = new PlotModel { Title = title };
    model.Axes.Add(new LinearAxis { Position = AxisPosition.Bottom, Title = "Індекс елемента", IsPanEnabled = false,
        IsZoomEnabled = false });
    model.Axes.Add(new LinearAxis { Position = AxisPosition.Left, Title = "Значення", IsPanEnabled = false, IsZoomEnabled =

```



```

        false });

var dataSeries = new LineSeries
{
    Title = "Відсортовані дані",
    Color = OxyColors.DodgerBlue,
    MarkerType = MarkerType.None,
    CanTrackerInterpolatePoints = false,
    TrackerFormatString = "Індекс: {2}\nЗначення: {4}"
};

for (int i = 0; i < result.Data!.Length; i++) { dataSeries.Points.Add(new DataPoint(i, result.Data[i])); }
model.Series.Add(dataSeries);
plotView.Model = model;
plotView.InvalidatePlot();
txtLog.ScrollToEnd();
}

private void RunAnalysis(object sender, RoutedEventArgs e)
{
    InputWindow inputWin = new InputWindow("Введіть кількість тестових масивів:", "10") { Owner = this, Title =
        "Кількість масивів" };
    if (inputWin.ShowDialog() == true)
    {
        int arrayCount = inputWin.Count;
        txtLog.Clear();
        txtLog.AppendText($"> Запускаю {arrayCount} масивів...\n");
        txtLog.AppendText("> Зачекайте, виконуються обчислення...\n");
        Mouse.OverrideCursor = Cursors.Wait;
        Application.Current.Dispatcher.Invoke(() => { }, System.Windows.Threading.DispatcherPriority.Render);

        try
        {
            bool showOperations = mnuShowOperations.IsChecked == true;
            var model = new PlotModel { Title = showOperations ? "Аналіз кількості операцій" : "Аналіз часу виконання" };
            model.Legends.Add(new OxyPlot.Legends.Legend
            {
                LegendPosition = OxyPlot.Legends.LegendPosition.TopLeft,
                LegendPlacement = OxyPlot.Legends.LegendPlacement.Inside,
                LegendBackground = OxyColor.FromAColor(220, OxyColors.White),
                LegendBorder = OxyColors.Black
            });
            model.Axes.Add(new LinearAxis { Position = AxisPosition.Bottom, Title = "Розмір масиву (N)", IsPanEnabled = false,
                IsZoomEnabled = false });
            model.Axes.Add(new LinearAxis { Position = AxisPosition.Left, Title = showOperations ? "Кількість операцій" : "Час
                виконання (мс)", IsPanEnabled = false, IsZoomEnabled = false });
            var dataSeries = new LineSeries
            {
                Title = showOperations ? "Кількість операцій" : "Практичний час (мс)",
                CanTrackerInterpolatePoints = false,

```

```

        Color = showOperations ? OxyColors.DodgerBlue : OxyColors.Lime,
        MarkerType = showOperations ? MarkerType.Triangle : MarkerType.Circle
    };
    LineSeries theorySeries = null;
    if (showOperations)
    {
        theorySeries = new LineSeries { Title = "Теоретична складність  $O(n^2)$ ", CanTrackerInterpolatePoints = false,
        Color = OxyColors.Gray, LineStyle = LineStyle.Dash };
    }

    List<SortResult> results = _appManager.RunFullAnalysis(arrayCount);
    double totalTime = 0;

    for (int i = 0; i < results.Count; i++)
    {
        int n = results[i].Data!.Length;
        double time = results[i].TimeMs;
        long ops = results[i].Operations;
        totalTime += time;

        txtLog.AppendText($"[{Macus {i + 1}}] Розмір: N = {n}, Час: {time:F3} мс, Операцій: {ops}\n");
        if (showOperations)
        {
            dataSeries.Points.Add(new DataPoint(n, ops));
            theorySeries.Points.Add(new DataPoint(n, Math.Pow(n, 2) / 3.0));
        }
        else dataSeries.Points.Add(new DataPoint(n, time));
    }

    txtLog.AppendText(">> -----\n");
    txtLog.AppendText($">> Загальний час сортування: {totalTime / 1000.0:F2} с ({totalTime:F0} мс)\n");
    txtLog.AppendText($">> Середній час: {(totalTime / results.Count):F3} мс\n");
    model.Series.Add(dataSeries);
    if (showOperations) model.Series.Add(theorySeries);
    plotView.Model = model;
    plotView.InvalidatePlot();
    txtLog.ScrollToEnd();
}
finally {
    Mouse.OverrideCursor = null;
    txtLog.ScrollToEnd();
}
}
}

private void OpenInputWindow(object sender, RoutedEventArgs e)
{
    InputWindow inputWin = new InputWindow("Введіть кількість елементів у масиві:", "1000") { Owner = this, Title =
    "Генерація масиву" };
    if (inputWin.ShowDialog() == true)

```

```

    {
        txtLog.AppendText($"> Генерую {inputWin.Count} елементів...\n");
        Mouse.OverrideCursor = Cursors.Wait;
        Application.Current.Dispatcher.Invoke(() => { }, System.Windows.Threading.DispatcherPriority.Render);

        try
        {
            SortResult result = _appManager.GenerateAndSort(inputWin.Count);
            ProcessSingleArray(result, $"Візуалізація масиву (N={inputWin.Count})");
        }
        finally { Mouse.OverrideCursor = null; }
    }
}

private void ExitApp(object sender, RoutedEventArgs e) { Application.Current.Shutdown(); }
}
}

```

Вміст файлу InputWindow.xaml:

```

<Window x:Class="LR2.InputWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:LR2"
        mc:Ignorable="d"
        Title="Введення" WindowStartupLocation="CenterOwner" ResizeMode="NoResize" SizeToContent="WidthAndHeight"
        MinWidth="250" Icon="/icons8-info-120.png">
    <StackPanel Margin="10">
        <TextBlock x:Name="txtPrompt" Text="Введіть значення:"/>
        <TextBox x:Name="txtInput" Margin="0,10, 0, 10" Text="1000"/>
        <Button Content="OK" Click="Ok_Click" IsDefault="True"/>
    </StackPanel>
</Window>

```

Вміст файлу InputWindow.cs:

```

using System.Windows;

namespace LR2
{
    public partial class InputWindow : Window
    {
        public InputWindow(string promptMessage, string defaultInput)
        {
            InitializeComponent();
            txtPrompt.Text = promptMessage;
            txtInput.Text = defaultInput;
        }

        public int Count { get; private set; }
    }
}

```

```

private void Ok_Click(object sender, RoutedEventArgs e)
{
    if (int.TryParse(txtInput.Text, out int result) && result > 0)
    {
        Count = result;
        DialogResult = true;
    }
    else MessageBox.Show("Введіть ціле додатне число!");
}
}
}

```

Вміст файлу ISorter.cs:

```

namespace LR2.SortingLogic
{
    public interface ISorter
    {
        SortResult Sort(int[] input);
    }
}

```

Вміст файлу ShakerSorter.cs:

```

using System.Diagnostics;

namespace LR2.SortingLogic
{
    public class ShakerSorter : ISorter {
        public SortResult Sort(int[] input) {
            if (input == null || input.Length == 0)
                throw new ArgumentException("Помилка: Масив порожній.");

            int[] m = (int[])input.Clone();
            long ops = 0;
            int n = m.Length;
            int left = 1, right = n - 1, j, t, k = n - 1;

            Stopwatch sw = Stopwatch.StartNew();
            do {
                for (j = right; j >= left; j--) {
                    ops++;
                    if (m[j - 1] > m[j]) { t = m[j - 1]; m[j - 1] = m[j]; m[j] = t; k = j; }
                }
                left = k + 1;
                for (j = left; j <= right; j++) {
                    ops++;
                    if (m[j - 1] > m[j]) { t = m[j - 1]; m[j - 1] = m[j]; m[j] = t; k = j; }
                }
                right = k - 1;
            }
            while (left <= right);
            sw.Stop();
        }
    }
}

```

```

        return new SortResult { TimeMs = sw.Elapsed.TotalMilliseconds, Operations = ops, Data = m };
    }
}
}

```

Вміст файлу SortingManager.cs:

```

namespace LR2.SortingLogic
{
    public class SortingManager
    {
        private readonly ISorter _sorter;
        private int[]? _currentArray;

        public SortingManager(ISorter sorter) { _sorter = sorter; }
        public int[]? CurrentArray => _currentArray;

        public SortResult GenerateAndSort(int size)
        {
            int[] rawData = DataHelper.Generate(size);
            SortResult result = _sorter.Sort(rawData);
            _currentArray = result.Data;
            return result;
        }

        public SortResult LoadAndSort(string path)
        {
            _currentArray = DataHelper.Load(path);
            SortResult result = _sorter.Sort(_currentArray);
            _currentArray = result.Data;
            return result;
        }

        public void SaveCurrent(string path)
        {
            if (_currentArray == null) throw new InvalidOperationException("Немає даних для збереження.");
            DataHelper.Save(path, _currentArray);
        }

        public List<SortResult> RunFullAnalysis(int arrayCount)
        {
            var results = new List<SortResult>();
            for (int i = 1; i <= arrayCount; i++)
            {
                int n = i * 1000;
                int[] data = DataHelper.Generate(n);
                results.Add(_sorter.Sort(data));
            }
            return results;
        }
    }
}

```

```
}
```

Вміст файлу SortResult.cs:

```
namespace LR2.SortingLogic
```

```
{
```

```
    public class SortResult
```

```
    {
```

```
        public double TimeMs { get; set; }
```

```
        public long Operations { get; set; }
```

```
        public int[]? Data { get; set; }
```

```
    }
```

```
}
```

Вміст файлу DataHelper.cs:

```
using System.IO;
```

```
namespace LR2.SortingLogic
```

```
{
```

```
    public static class DataHelper
```

```
    {
```

```
        private static Random rnd = new Random();
```

```
        public static int[] Generate(int size)
```

```
        {
```

```
            if (size <= 0) throw new ArgumentException("Розмір має бути більшим за 0.");
```

```
            return Enumerable.Range(0, size).Select(_ => rnd.Next(-10000, 10000)).ToArray();
```

```
        }
```

```
        public static void Save(string path, int[] data)
```

```
        {
```

```
            if (data == null || data.Length == 0) throw new ArgumentException("Немає даних.");
```

```
            File.WriteAllText(path, string.Join(" ", data));
```

```
        }
```

```
        public static int[] Load(string path)
```

```
        {
```

```
            if (!File.Exists(path)) throw new FileNotFoundException("Файл не знайдено.");
```

```
            string content = File.ReadAllText(path);
```

```
            if (string.IsNullOrWhiteSpace(content)) throw new ArgumentException("Файл порожній.");
```

```
            var parts = content.Split(new[] { ' ', '\n', '\r', ';' }, StringSplitOptions.RemoveEmptyEntries);
```

```
            int[] result = new int[parts.Length];
```

```
            for (int i = 0; i < parts.Length; i++)
```

```
            {
```

```
                if (!int.TryParse(parts[i], out result[i])) throw new FormatException($"Елемент '{parts[i]}' не є цілим числом. Перевірте файл.");
```

```
            }
```

```
            return result;
```

```
        }
```

```
    }
```

```
}
```

